



Université de Rouen Normandie

SSL Checker
Plan de Développement

Plan de Développement

Étudiants :
Mehdi LASMI
Marzouk MONIR
Malik MAHFOUDI
Karina AIT-ABDELKAOUI
Naby TRAORE SAVANE
Rayan BOUGHANEM
Hamza RAHMOUNI
Amr ELGOHARY

Client :
Bruno MACADRÉ

Table des matières

1 Contexte du projet **3**
- 1.1 Origine du projet . . . 3
- 1.2 Contexte du développement . . . 3
- 1.3 Principaux acteurs . . . 3
- 1.4 Objectifs poursuivis . . . 3
- 1.5 Documents de référence . . . 3

2 Méthodologie de Développement **4**
- 2.1 Étapes de la Méthodologie . . . 4
  - 2.1.1 Création de la vision produit . . . 4
  - 2.1.2 Planification des sprints . . . 4
  - 2.1.3 Exécution du sprint . . . 5
  - 2.1.4 Revue de sprint : à la fin de chaque sprint, une Sprint Review est organisée pour : . . . 5
  - 2.1.5 Rétrospective du sprint . . . 5
  - 2.1.6 Livraison itérative et tests utilisateurs . . . 5
- 2.2 Avantages de Scrum . . . 6

3 Organisation et Responsabilités de l'équipe **7**
- 3.1 La matrice RACI . . . 8

4 Organigramme des Tâches **9**
- 4.1 Définition des Produits Intermédiaires . . . 9
  - 4.1.1 Infrastructure et Environnement d'Exécution . . . 9
  - 4.1.2 Moteur d'Analyse et de SSLChecker . . . 9
  - 4.1.3 Les Modules de Scan . . . 9
  - 4.1.4 Notation (Scoring) . . . 10
  - 4.1.5 Rapport . . . 10
- 4.2 Diagramme Hiérarchique . . . 11
- 4.3 Répartition des tâches et charges de travail . . . 11
  - 4.3.1 Note de synthèse sur la répartition des charges . . . 11

5 Besoins en moyens et en ressources **12**
- 5.1 Ressources matérielles et logicielles . . . 12
- 5.2 Environnement de développement et Collaboration . . . 13
- 5.3 Environnement technique et Déploiement . . . 13

6 Planning général **14**
6.1 Dates clés et livrables principaux 14
6.2 Organisation des itérations 14

7 Procédés de gestion **15**
7.1 Gestion de la documentation 15
7.2 Gestion des configurations 15

8 Revues et points clefs **16**

9 Procédure de suivi d'avancement **17**
9.1 Suivi interne 17
9.2 Suivi externe 17

A Annexes **18**
A.1 Organigramme des tâches (WBS) 18
A.2 Diagrammes Gantt des sprints 19

Table des figures

1 Organigramme des tâches du projet 18
2 Diagramme Gantt du Sprint 1 19
3 Diagramme Gantt du Sprint 2 20
4 Diagramme Gantt du Sprint 3 21

Liste des tableaux

1 Rôles et responsabilités de l'équipe 7
2 La matrice RACI 8
3 Répartition des tâches et charges de travail 11
4 Synthèse des dates clés de la livraison des versions de l'application 14

1 Contexte du projet

1.1 Origine du projet

Notre projet *SSL Checker* est réalisé dans le cadre du cours de gestion de projet. Le but du projet est de développer un outil web intranet permettant de tester la qualité SSL/TLS d'un serveur web. Cette initiative nous offre l'opportunité de bénéficier d'une expérience pratique dans le domaine de la sécurité informatique.

1.2 Contexte du développement

Notre projet est mené dans un cadre pédagogique, avec des ressources et un temps limités. Cette situation encourage le travail en équipe, avec la définition des rôles, la répartition des tâches et la coordination entre les membres de notre équipe.

1.3 Principaux acteurs

Ce projet est realisé par notre équipe de huit étudiants, encadrée par le client M.MACADRÉ, et supervisée par le chargé de cours M.VAUTRAIN ainsi que le chargé des travaux pratiques M.GODARD du module de gestion de projet.

1.4 Objectifs poursuivis

Notre équipe doit développer un outil capable de scanner des serveurs SSL/TLS et de produire un rapport synthétique. Par ailleurs, le projet nous permet de comprendre le protocole SSL/TLS et d'acquérir les principes fondamentaux de la sécurité web.

1.5 Documents de référence

Les références utilisées incluent les supports du cours, les standards SSL/TLS, la documentation des bibliothèques Python ainsi que des outils de référence pour l'évaluation et l'analyse de la sécurité, par exemple : SSL Labs.

2 Méthodologie de Développement

Scrum est un **cadre de travail (framework)** qui aide les équipes à faire face à des problèmes **complexes et changeants**, tout en livrant régulièrement des produits **à forte valeur ajoutée**.
Scrum repose sur le **travail en équipe**, l'**amélioration continue** et des **livraisons fréquentes** sous forme d'itérations courtes appelées *sprints*. Chaque livraison apporte un **incrément du produit**, utilisable et améliorable.
Ce Framework est basé sur le **contrôle empirique des processus**, c'est-à-dire sur l'apprentissage par l'expérience, l'observation des résultats et l'adaptation continue afin de mieux répondre aux besoins des clients.

2.1 Étapes de la Méthodologie

2.1.1 Création de la vision produit

Collecte des besoins : Le client (M.MACADRÉ) a exprimé ses besoins ainsi que les fonctionnalités attendues de l'outil SSLChecker.

Récits utilisateurs (user stories) :

- Les besoins techniques ont été transformés en récits utilisateurs afin d'identifier clairement les fonctionnalités à développer.
- Chaque user story inclut des critères d'acceptation définissant précisément le comportement attendu de l'outil.
- Exemple : *"En tant qu'administrateur système, je veux scanner mon nom de domaine afin de savoir si mon certificat est expiré ou sur le point d'être."*

2.1.2 Planification des sprints

Sprint Planning Meeting : Avant chaque sprint, l'équipe se réunit pour :

- Prioriser les fonctionnalités présentes dans le backlog produit
- Définir les objectifs du sprint en fonction de la capacité de l'équipe

Durée des sprints :
Chaque sprint dure 3 semaines, ce qui permet une livraison rapide des versions de l'application web.

2.1.3 Exécution du sprint

Réunion hebdomadaire d'équipe : Réunion de 15 à 30 minutes chaque semaine pour :

- Identifier les blocages techniques
- Ajuster les priorités de développement si nécessaire

Développement des fonctionnalités :

- Les développeurs implémentent les user stories sélectionnées en respectant les critères d'acceptation
- Des tests unitaires sont systématiquement mis en place afin de garantir l'exactitude des diagnostics et d'éviter les faux positifs

2.1.4 Revue de sprint : à la fin de chaque sprint, une Sprint Review est organisée pour :

- Présenter au client l'interface et les fonctionnalités développées
- Vérifier que les user stories livrées respectent les critères d'acceptation
- Valider que les tests de sécurité sont conformes aux normes en vigueur

2.1.5 Rétrospective du sprint

À l'issue de chaque sprint, une Sprint Rétrospective est réalisée afin d'analyser :

- Les éléments ayant bien fonctionné durant le sprint
- Les problémes rencontrés
- Les axes d'amélioration pour optimiser le développement et l'éfficacité de l'équipe lors des prochains sprints

2.1.6 Livraison itérative et tests utilisateurs

- Une version fonctionnelle de l'outil est livrée au client à la fin de chaque sprint
- Une version complète est testée à l'aide de scénarios réels et comparée à outils similaires afin d'éviter les faux résultats

2.2 Avantages de Scrum

- **Adaptabilité : Les priorités peuvent être modifiées en fonction des retours du client après chaque sprint**
- **Transparence : Le client est consulté régulièrement afin de garantir que le produit final répond à ses attentes**
- **Amélioration continue : Les rétrospectives permettent d'identifier et de corriger les points faibles à chaque étape du développement**

3 Organisation et Responsabilités de l'équipe

Notre équipe est organisée selon le modèle Scrum avec trois rôles clés, adaptés aux besoins techniques de l'application :

Rôle	**Responabilités**	**Personne en charge**
Product Owner	Gérer la vision du produit. Définir les priorités de sécurité . Valider les user stories techniques	Rayan BOUGHANEM
Scrum Master	Garantir le respect des rituels Scrum. Lever les obstacles techniques liés aux environnements de test et serveurs	Karina AIT-ABDELKAOULI
Équipe de Développement	Développer le moteur de l'app et les differents fonctions(Python / OpenSSL). Implémenter les tests de conformité SSL/TLS. Gérer le versioning via GitLab/GitHub	Mehdi LASMI Marzouk MONIR Malik MAHFOUDI Naby TRAORE SAVANE Hamza RAHMOUNI Amr ELGOHARY

TABLE 1 – Rôles et responsabilités de l'équipe

3.1 La matrice RACI

Tâche	Réalisation	Approbation	Informé
Configuration	Rayan,Naby, Amr	Rayan	toute l'équipe
Architecture logicielle	Malik, Hamza, Mehdi	Rayan	toute l'équipe
Frontend	Karina,Marzouk	Rayan	toute l'équipe
Module Certificat	Naby, Hamza	Rayan	toute l'équipe
Module Protocole	Malik	Rayan	toute l'équipe
Module Cipher	Mehdi, Marzouk	Rayan	toute l'équipe
Module Échange de clés	Karina	Rayan	toute l'équipe
Module Vulnérabilité	Amr	Rayan	toute l'équipe
Module Scan de port	Hamza	Rayan	toute l'équipe
Module Compatibilité OS	Marzouk	Rayan	toute l'équipe
Module Note	Rayan	-	toute l'équipe
Module Rapport	Rayan	-	toute l'équipe
Revue de code	toute l'équipe	Rayan	Client + l'équipe
Livraison de la version	Rayan	Client	Client + l'équipe

TABLE 2 – La matrice RACI

Remarques

Pour les réalisateurs, le premier nom indiqué correspond à la personne qui commence la tâche. Les noms suivants désignent les personnes qui interviennent ensuite pour aider, une fois qu'elles ont terminé leur propre partie du même sprint qui est plus courte.
Pour prévenir un risque de retard ou de mauvaise organisation, si l'approbateur n'est pas disponible, le premier réalisateur en charge de la tâche assume, au maximum, la responsabilité de l'approbateur.

4 Organigramme des Tâches

L'Organigramme des Tâches (WBS) constitue la structure directrice du projet. Il décompose le produit final SSL Checker en unités gérables et autonomes, garantissant que chaque exigence technique est traduite en une action concrète.

4.1 Définition des Produits Intermédiaires

4.1.1 Infrastructure et Environnement d'Exécution

Ce domaine représente le socle technique du projet. L'objectif est de fournir un environnement isolé et sécurisé via la conteneurisation (Docker). Cela garantit que l'outil est déployable sur une VM Linux en intranet sans conflit de dépendances. La gestion des flux est assurée par un reverse-proxy (Nginx) et une interface de passerelle applicative (Gunicorn). Ce travail est couvert par la tâche **Fondations**, qui inclut les éléments suivants :

- **La configuration** : cette partie concerne :
  - la préparation et la configuration de la machine virtuelle Linux
  - la configuration du reverse-proxy Nginx
  - le paramétrage du serveur WSGI (Gunicorn)
  - la gestion et la configuration des conteneurs Docker
- **L'architecture logicielle**, qui constitue le squelette de l'application

4.1.2 Moteur d'Analyse et de SSLChecker

Le moteur est l'intelligence centrale de l'outil, conçu selon une architecture modulaire et orientée objet. Ce choix technique permet à chaque composant d'être totalement autonome : il gère ses propres tests, valide ses données et retourne un objet consolidé au contrôleur principal.

4.1.3 Les Modules de Scan

- **Module Certificat :** Responsable de l'extraction complète des données X.509. Il valide la chaîne de certification, vérifie la robustesse des signatures et interroge les répondeurs OCSP afin de s'assurer que le certificat n'est pas révoqué. La vérification du module est réalisée après le développement du module.

— **Module Protocole :** Effectue des tentatives de connexion forcées pour identifier toutes les versions supportées (du SSL 2.0 au TLS 1.3). Il détermine précisément si le serveur accepte des protocoles jugés obsolètes. La vérification du module est réalisée après le développement du module.
— **Module Cipher :** Ce module assure l'énumération des suites de chiffrement supportées par le serveur. En plus de cette liste, il exécute des tests comparatifs pour identifier l'ordre de préférence (Cipher Order). Cela permet de déterminer si le serveur impose ses propres algorithmes les plus sécurisés ou s'il suit aveuglément le choix du client. La vérification du module est réalisée après le développement du module.
— **Module Échange de Clés :** Analyse spécifiquement la solidité des paramètres de sécurité lors de la phase de négociation, notamment pour les clés RSA et les échanges Diffie-Hellman (DH/ECDH). La vérification du module est réalisée après le développement du module.
— **Module Vulnérabilité :** Orchestre des tests ciblés sur des failles critiques (Heartbleed, POODLE, SWEET32) et vérifie l'implémentation des mécanismes de protection comme le HSTS. La vérification du module est réalisée après le développement du module.
En complément des analyses SSL, le moteur effectue un scan des ports ouverts pour identifier les services exposés, et aussi vérifier la compatibilité des systèmes d'exploitation (OS) supportés par la configuration actuelle du serveur.

4.1.4 Notation (Scoring)

Une fois que chaque module a terminé son travail de manière indépendante, les résultats sont agrégés pour calculer une Note globale de sécurité,permettant d'évaluer précisément la configuration du serveur. Ce travail est couvert par la tâche **Module Note**.

4.1.5 Rapport

Une fois l'analyse terminée, les résultats sont mis en forme afin d'être transmis au frontend. Ce travail est couvert par la tâche **Module Rapport**.

4.2 Diagramme Hiérarchique

Le diagramme hiérarchique (WBS) détaillant la décomposition du projet a été élaboré et validé par l'équipe. Il est présenté en annexe.

4.3 Répartition des tâches et charges de travail

Tâche	Fonctionnalité	Priorité	Charge (h)
01	Fondations	Indispensable	10
02	Module Certificat	Indispensable	15
03	Module Protocole	Indispensable	10
04	Module Cipher	Indispensable	15
05	Module Échange de clés	Indispensable	12
06	Module Vulnérabilité	Indispensable	15
07	Module Note	Indispensable	6
08	Frontend	Indispensable	7
09	Module Compatibilité OS	Optionnel	4
10	Module Scan de ports	Optionnel	4
11	Module Rapport	Optionnel	4
12	Revue de code et livrable	Indispensable	5
TOTAL			**107**

TABLE 3 – Répartition des tâches et charges de travail

Remarque

D'après la répartition des tâches, la charge totale est de 107 heures par sprint. Selon les estimations, chaque membre de l'équipe dispose d'environ 6 heures de travail par semaine donc un total de 144 heures par sprint. Ainsi, nous avons une marge de 37 heures par sprint, qui sert de mesure de sécurité face à un risque potentiel de retard.

4.3.1 Note de synthèse sur la répartition des charges

La répartition des heures a été organisée pour mettre l'accent sur les parties les plus importantes de l'outil. L'objectif est de consacrer le maximum de temps aux tests de sécurité tout en gardant une application facile à utiliser. Le projet est ainsi divisé en quatre grandes parties techniques.

1. **Le Cœur de Scan (72% de la charge - 77h)** : C'est la pièce maîtresse du projet. Elle regroupe les modules Certificat, Protocoles, Ciphers, Échange de clés et Vulnérabilités. L'effort est concentré sur cette partie pour garantir la fiabilité et la précision des analyses de sécurité.
2. **L'Infrastructure et l'Architecture (21% de la charge - 22h)** : Ce pilier assure la robustesse et la portabilité du système. Il comprend la préparation de la VM, la configuration de Docker, du reverse-proxy Nginx, du serveur Gunicorn ainsi que la revue de code finale pour valider l'architecture logicielle.
   Cette partie sert à transformer les données brutes en informations faciles à lire. Elle regroupe le calcul de la note de sécurité (Scoring), l'affichage sur le site web et la création de rapport de synthèse. Le but est de donner à l'utilisateur un résumé clair pour qu'il sache tout de suite si son serveur est bien protégé ou s'il y a des problèmes à corriger.
3. **Diagnostic Réseau Complémentaire (7% de la charge - 8h)** : Ces tâches enrichissent l'audit par une vision globale de la cible. Le Scan de ports et la Compatibilité OS apportent un contexte technique utile qui vient compléter l'analyse spécifique aux couches SSL/TLS.

5 Besoins en moyens et en ressources

Pour assurer le bon déroulement du projet SSL Checker et garantir une expérience de développement optimale, les ressources suivantes ont été mobilisées :

5.1 Ressources matérielles et logicielles

Postes de travail : Chaque membre de l'équipe dispose d'un ordinateur (8 Go RAM, i5 minimum) et d'une connexion internet stable pour la collaboration sur GitLab.

Bibliothèques de sécurité (Cœur du moteur) :

— **Cryptography :** Dédiée au parsing profond des certificats X.509 (champs SAN, EKU et validation de la chaîne de confiance).

— **OpenSSL & Scapy :** Utilisés en complémentarité pour l'analyse des Cipher Suites. Scapy est crucial pour forger manuellement des paquets,

permettant ainsi de tester des protocoles obsolètes (SSLv2/v3) que les bibliothèques standards du système bloqueraient par sécurité.

5.2 Environnement de développement et Collaboration

IDE : Utilisation de Visual Studio Code pour son intégration native avec Python et la gestion des conteneurs.

Gestion de projet :

- **GitLab :** Centralisation du code source, gestion du versioning et des Merge Requests
- **Trello :** Suivi des tâches en mode Agile pour respecter le planning
- **WhatsApp :** Pour échanger rapidement et discuter facilement entre les membres de l'équipe
- **Discord :** Utilisé pour les discussions directes avec le client afin de suivre l'avancée du projet

5.3 Environnement technique et Déploiement

Docker & Docker-Compose : Outils essentiels pour organiser et faire fonctionner les différents services (Frontend, Backend, Nginx). Ils permettent de garantir que l'application marche exactement de la même manière sur l'ordinateur des développeurs et sur le serveur final, évitant ainsi les erreurs d'installation.

6 Planning général

Le planning général du projet permet de comprendre l'enchaînement global des itérations sans entrer dans le détail des tâches techniques.

Le projet est structuré en plusieurs **itérations (sprints)**, chacune visant la livraison d'une version de l'application. Les diagrammes détaillés de chaque sprint sont fournis en **annexes** sous forme de diagrammes de Gantt.

6.1 Dates clés et livrables principaux

Phase / Événement	**Durée**	**Livrable associé**
Sprint 1	08/02/2026-28/02/2026	Livraison de la version 1
Sprint 2	01/03/2026-21/03/2026	Livraison de la version 2
Sprint 3	22/03/2026-11/04/2026	Livraison de la version 3

TABLE 4 – Synthèse des dates clés de la livraison des versions de l'application

Remarques

Le diagramme de Gantt a été choisi car il offre une vue claire et intuitive du planning et permet de visualiser les deadlines fixées pour chaque tâche.
Une marge a été créée entre la fin du troisième sprint et la date finale de livraison de l'application. Cette marge a pour objectif de prévoir un espace permettant de gérer d'éventuels retards pouvant survenir durant le développement ou tout autre risque majeur.

6.2 Organisation des itérations

Le projet est organisé en **sprints de trois semaines**. Chaque sprint est dédié au **développement et aux tests** et se conclut par la livraison d'une **nouvelle version fonctionnelle de l'application**.

7 Procédés de gestion

7.1 Gestion de la documentation

Pendant le projet, plusieurs documents seront produits pour assurer le suivi et la traçabilité des travaux. Les documents seront versionnés et centralisés pour garantir leur accessibilité et leur historique.

Les règles principales de production et de gestion des documents sont :

— Versionnement de chaque document
— Stockage centralisé dans un espace partagé
— Validation avant diffusion

7.2 Gestion des configurations

Pour assurer la maîtrise des fichiers et des versions pendant le développement :

Nomenclature claire : pour assurer la cohérence et la traçabilité des fichiers pendant le développement, la nomenclature est définie selon les règles suivantes :

— Les mots sont attachés en CamelCase.
— Préfixer le nom par le type de fichier. Exemple : `modeleXXX`.
— Préciser la fonction ou le contenu du fichier. Exemple : `modeleProtocole`.

Organisation structurée des espaces de travail : l'organisation des fichiers suit une hiérarchie claire, facilitant le développement collaboratif et la maintenance. Le code source est organisé selon l'architecture MVC.

projetSSLChecker/
src/ *(architecture MVC)*
modele/ ← logique métier (Modèle)
vue/ ← affichage des résultats (Vue)
controleur/ ← orchestration des modules (Contrôleur)
SSLChecker.py
tests/
docs/

Sauvegardes régulières : afin de garantir la continuité du projet et d'éviter toute perte de données, des sauvegardes régulières sont effectuées.
Pour le code source et les tests, un système de gestion de versions **Git** est utilisé, avec des branches distinctes pour le développement `dev`(version en cours de developpement) et la version stable `main`(la dernière version validée). Chaque modification est enregistrée avec un message descriptif, ce qui permet de suivre l'historique complet et d'identifier qui a effectué chaque changement.
Pour les documents, des copies sont régulièrement sauvegardées dans un dossier partagé accessible à toute l'équipe.
La fréquence des sauvegardes se fait après chaque modification majeure.

Validation des changements majeurs avant intégration : Pour garantir la qualité et la stabilité de la version principale du projet `main`, toute modification majeure doit être validée avant d'être intégrée. Cette validation consiste à :

- Vérifier que le code a été testé et fonctionne correctement dans la branche de développement `dev`
- Confirmer que les modifications respectent la nomenclature et l'organisation des fichiers définies
- S'assurer que tous les tests unitaires et fonctionnels associés passent sans erreur
- Réaliser une revue de code pour valider la logique et la qualité du code

Une fois ces vérifications effectuées, les changements sont fusionnés *merge* dans la branche stable `main`.

8 Revues et points clefs

Pour assurer le suivi du projet, des revues sont prévues à la fin de chaque sprint. Chaque revue permet de vérifier que les développements et les tests ont été correctement réalisés et que la version livrée est fonctionnelle.

Toutes les revues sont réalisées par l'ensemble de l'équipe afin de garantir une validation collective et un suivi cohérent du projet.

9 Procédure de suivi d'avancement

L'avancement du projet est suivi tout au long des sprints afin d'assurer la coordination de l'équipe et la qualité des livrables.

9.1 Suivi interne

Le suivi interne consiste à organiser des réunions hebdomadaires durant les sprints, pour assurer l'avancement des tâches, vérifier les tests réalisés et identifier les éventuels blocages. L'avancement est également mis à jour sur les outils de suivi collaboratif : Trello, GitLab, permettant à chaque membre de visualiser l'état des tâches, suivre les priorités et respecter les deadlines. Cette organisation favorise la communication interne, responsabilise les membres de l'équipe et assure la traçabilité des décisions et actions.

9.2 Suivi externe

Le suivi externe permet de tenir le le client informé de l'état d'avancement et de la qualité du projet, tout en assurant la transparence des actions de l'équipe.

Des rapports sont préparés à la fin de chaque sprint. Ces rapports présentent : - l'avancement des modules développées - les tests réalisés et leurs résultats - les difficultés rencontrées ou les risques identifiés, ainsi que les actions prévues pour les résoudre Les rapports sont remis au client sous un format simple, tel que PDF ou document partagé, afin de faciliter leur lecture et le suivi du projet. Cette approche permet au client d'avoir une vision globale du projet et de donner un retour constructif si nécessaire.

A Annexes

A.1 Organigramme des tâches (WBS)

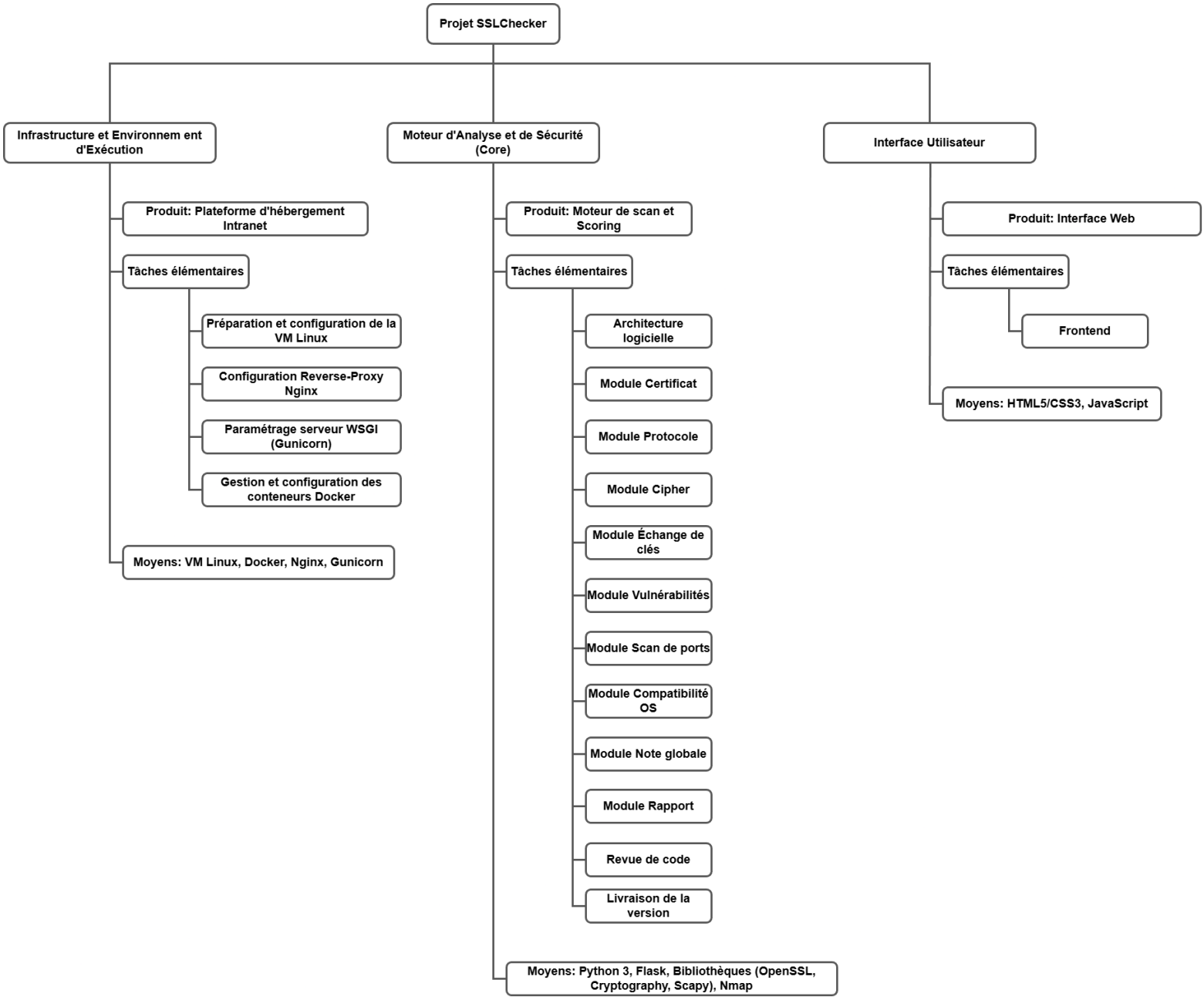


FIGURE 1 – Organigramme des tâches du projet

A.2 Diagrammes Gantt des sprints



FIGURE 2 – Diagramme Gantt du Sprint 1



FIGURE 3 – Diagramme Gantt du Sprint 2

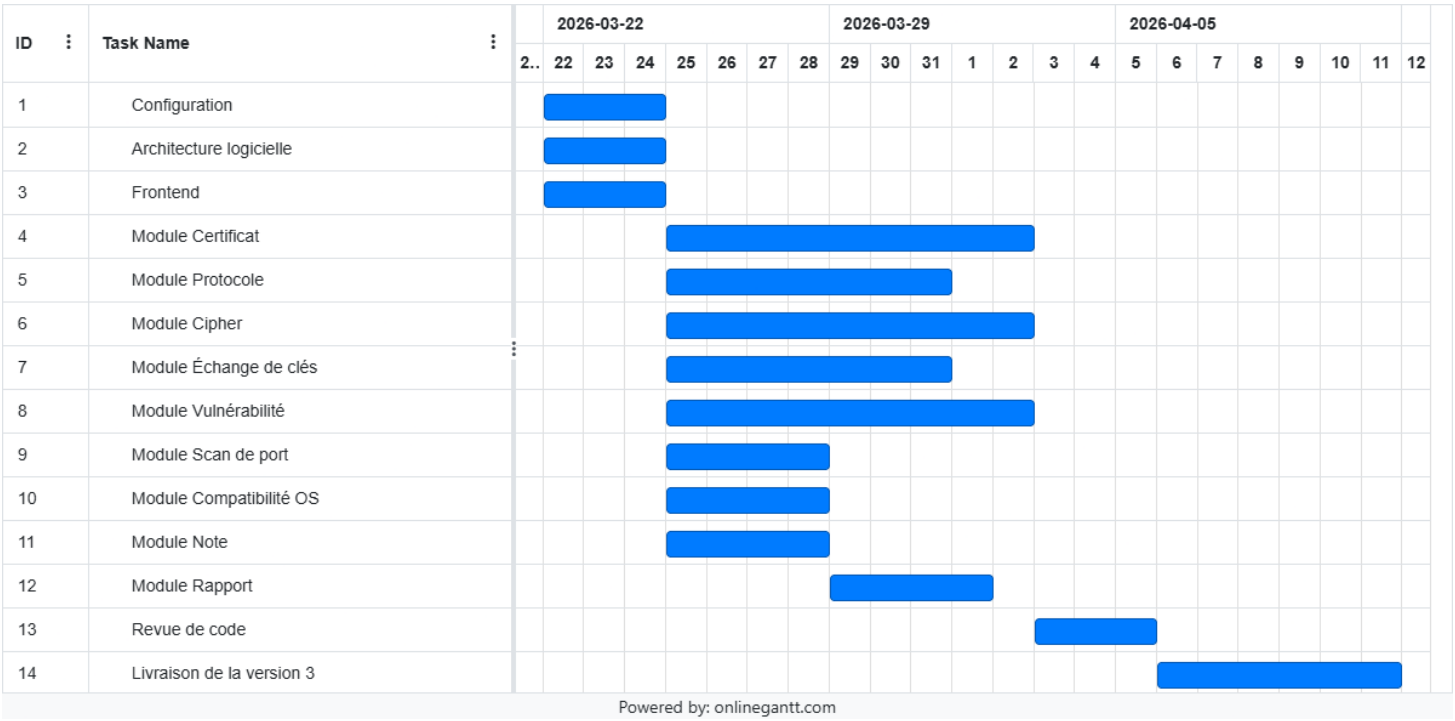


FIGURE 4 – Diagramme Gantt du Sprint 3

Bruno Macadre

